

Разработка классов в рамках игры «Герой и Магия»

Тема: Классы, методы экземпляра и декораторы.

Вы разрабатываете движок для простой текстовой RPG-игры. У вас есть класс Hero (Герой), который может сражаться и лечиться. Однако «голый» класс неудобно отлаживать и легко сломать (например, мертвый герой не должен атаковать).

Ваша задача состоит в улучшении класса Hero, используя **декораторы**, чтобы добавить проверку состояния и логирование действий, не меняя внутреннюю логику самих методов боя.

Задание

1. Создайте класс Hero

У класса должны быть следующие атрибуты и методы:

Атрибуты (в `__init__`):

- `name` (имя героя, строка)
- `health` (здоровье, целое число, изначально 60 у волшебника и 100 у воина)
- `mana` (мана, целое число, изначально 50 у волшебников и 10 у воина)
- `hero_class` – класс героя, волшебник или воин.
- `spells_names` – изученные заклинания. Словарь, содержащий названия заклинаний в качестве ключа, в качестве значения будет словарь, содержащий ключи `mana_cost` - затраты маны на одно использование, `attack_damage` – количество урона врагу от его использования, `health_increase` – количество здоровья, на которое заклинание восстанавливает здоровье героя.
- `items` – предметы, надетые на героя (максимально может быть 6 предметов), словарь, содержащий в качестве ключа название предмета, а значение – словарь, в качестве ключа параметр, на который накладывается эффект от предмета (здоровье или мана), значение – число, на которое нужно увеличить соответствующий параметр.

Методы:

- `attack(damage)`: отнимает здоровье у врага (просто выводит в консоль «Герой нанес урон: {damage}»).
- `heal(amount)`: восстанавливает здоровье герою (увеличивает `self.health`).
- `cast_spell(spell_name)`: тратит ману (уменьшает `self.mana`) и выводит название заклинания.
- `add_spell(spell_name, )`: добавляет новое заклинание в список.
- `add_item(item_name)`: добавляет новый предмет в список.

2. Напишите декоратор `@is_alive`

Этот декоратор должен проверять здоровье героя перед выполнением метода.

- Получите экземпляр класса (это будет первый аргумент в `*args`).
- Проверьте атрибут `health`.
- Если `health <= 0`: выведите сообщение «{name} мертв и не может действовать!» и не вызывайте оригинальный метод (верните `None`).
- Если `health > 0`: вызовите оригинальный метод.

3. Напишите декоратор `@log_action`

- Перед вызовом метода выведите: «[LOG] Начало действия: {имя_метода}».
 - После вызова метода выведите: «[LOG] Действие завершено».
 - Обязательно верните результат работы оригинального метода.
4. Примените декораторы
- Декоратор `@is_alive` на методы `attack` и `cast_spell`. (Герой не должен атаковать или колдовать, если умер).
 - Декоратор `@log_action` на метод `heal`. (Мы хотим следить, когда герой лечится).

Самостоятельная работа

В игре ивент, посвященный празднику пасхи. В связи с этим максимальное здоровье всех героев увеличено вдвое, манна увеличена в 1,5 раза. Используя декораторы, реализуйте функцию для увеличения здоровья на заданный период времени.

К празднику так же всем в предметы временно добавлен предмет священный посох, который добавляет всем волшебникам +5 к мане. Используя декораторы, реализуйте функцию для увеличения маны волшебника на заданный период времени.

Придумайте функционал, который вы бы реализовали в качестве декоратора. Реализуйте его. В коде сделайте строковую пометку (можно комментарий), что делает ваш декоратор.